

6.6 List nesting ⚙️ ^

Settings

Beta features

list nesting

Such embedding of a list inside another list is known as list nesting.

Figure 6.6.1: Multi-dimensional lists.

```
my_list = [[10, 20], [30, 40]]
print(f"First nested list: {my_list[0]}")
print(f"Second nested list: {my_list[1]}")
print(f"Element 0 of first nested list: {my_list[0][0]}")
```

First nested list: [10, 20]

Second nested list: [30, 40]

Element 0 of first nested list: 10

PARTICIPATION ACTIVITY

6.6.1: List nesting.

Start

☐ 2x speed

```
my_list = [ ["a", "b"], ["c", "d"], ["e", "f"] ]
```

my_list[0]

["a", "b"]

my_list[0][0] = "a"

my_list[1]

["c", "d"]

my_list[1][0] = "c"

my_list[2]

["e", "f"]

my_list[2][0] = "e"

`my_list[0][1] = "b"``my_list[1][1] = "d"``my_l`

Captions ^

1. The nested lists can be accessed using a single access operation.
2. The elements of each nested list can be accessed using two indexing operations.

multi-dimensional data structure

List nesting allows for a programmer to also create a multi-dimensional data structure, the simplest being a two-dimensional table, like a spreadsheet or tic-tac-toe board.

Figure 6.6.2: Representing a tic-tac-toe board using nested lists.

```
tic_tac_toe = [
    ["X", "O", "X"],
    [" ", "X", " "],
    ["O", "O", "X"]
]

print(tic_tac_toe[0][0], tic_tac_toe[0][1], tic_tac_toe[0][2])
print(tic_tac_toe[1][0], tic_tac_toe[1][1], tic_tac_toe[1][2])
print(tic_tac_toe[2][0], tic_tac_toe[2][1], tic_tac_toe[2][2])
```

PARTICIPATION ACTIVITY

6.6.3: Two-dimensional list.

Start ☐ 2x speed

```
my_list = [
    [10, 0, 55],
    [0, 4, 16]
]

# Write to some elements
my_list[0][0] = 33
my_list[1][1] = 77
my_list[1][2] = 99
```

Objects:

my_list

my_list[0]

my_list[1]

		Column	
		0	1
Rows [2]	0	33 [0][0]	0
	1	0 [1][0]	77

Captions ^

1. New list object contains other lists as elements.
2. Elements accessed by [row][column].

Try 6.6.1: Two-dimensional list example: Driving distance between cities.

The following example illustrates the use of a two-dimensional list in a distance between cities.

Run the following program, entering the text "1 2" as input to find the distance between LA and CL modifying the program by adding a new city, Anchorage, that is 3400, 3571, and 4551 miles from respectively.

Note that the styling of the nested list in this example makes use of indentation to clearly indicate spacing does not affect how the interpreter evaluates the list contents.



▶ Run

```
1  # direct driving distances between cities, in miles
2  # 0: Boston    1: Chicago    2: Los Angeles
3
4  distances = [
5      [
6          0,
7          960,    # Boston-Chicago
8          2960    # Boston-Los Angeles
9      ],
10     [
11         960,    # Chicago-Boston
12         0,
13         2011    # Chicago-Los Angeles
14     ],
15     [
16         2960,   # Los Angeles-Boston
17         2011,   # Los Angeles-Chicago
18         0
19     ]
20 ]
21
22 user_input = input("Enter city pair (Ex: 1 2) -- ").strip()
```

CONSOLE**Model Solution** ▼

Figure 6.6.3: The level of nested lists is arbitrary.

```
nested_table = [
    [
        [10, 0, 55],
        [0, 4, 16]
    ],
    [
        [0, 0, 1],
        [1, 20, 2]
    ]
]
```

nested for loops

A programmer can access all of the elements of nested lists by using nested for loops.

PARTICIPATION ACTIVITY

6.6.5: Iterating over multi-dimensional lists.

Start ☐ 2x speed

```
currency = [
    [1.00, 5.00, 10.0], # US Dollars
    [0.75, 3.77, 7.53], # Euros
    row --> [0.65, 3.25, 6.50] # British pounds
]

for row in currency:
    for cell in row:
        print(cell, end=" ")
    print()
```

1.00	5.00	10.00
0.75	3.77	7.53
0.65	3.25	6.50

Captions ^

1. Each iteration row is assigned the next list element from currency. Each item in a row is printed.

Figure 6.6.4: Iterating through multi-dimensional lists using enumerate().

```
currency = [  
    [1, 5, 10 ], # US Dollars  
    [0.75, 3.77, 7.53], #Euros  
    [0.65, 3.25, 6.50] # British pounds  
]  
for row_index, row in enumerate(currency):  
    for column_index, item in enumerate(row):  
        print(f"currency[{row_index}][{column_index}] is {item:.2f}")
```

**CHALLENGE
ACTIVITY**

6.6.1: List nesting.

763650.1722066.qx3zqy7

Start

Lists row1, row2, and row3 are read from input. Assign list_2d with a two-dimensional list consisting of row1, row2, and row3 in that order.

[Click here for example](#) ▼

```
1 row1 = input().split()
2 row2 = input().split()
3 row3 = input().split()
4
5 """ Your code goes here """
6
7 print(list_2d[0])
8 print(list_2d[1])
9 print(list_2d[2])
```

1

2

3

Check**Next level**

View solution ▼ *(Instructors only)*